TP2 pas à pas : construire un data lake

Enseignante : Safa Chmengui

Classe : Mastère Professionnel 2ème année

Présentation du TP2

Lien avec le TP1. Dans le TP1, on a construit un data warehouse pour le *Café Jasmin* : les données ont été nettoyées **avant** d'être chargées, dans des tables au schéma fixé à l'avance (*schema-on-write*). Ce modèle est fiable, mais il refuse tout ce qui n'entre pas dans un tableau.

Nouveau besoin. Le café se développe. Il reçoit maintenant des **avis clients** depuis son application (fichier JSON avec des notes et des commentaires libres) et des **journaux de livraison** produits par son application de coursiers (fichiers texte). Ces données ne rentrent pas dans le schéma en étoile du TP1. Le gérant veut pourtant les conserver toutes, à faible coût, pour les analyser plus tard.

La solution étudiée : le data lake. On stocke **tous** les fichiers dans leur format d'origine, sans les transformer, dans un espace organisé par source et par date. La structure n'est appliquée qu'au moment de la lecture (*schema-on-read*).

À la fin du TP, vous saurez :

- organiser un data lake en zones (`raw`, `processed`) rangées par source et par date ;
- ingérer des fichiers de formats différents sans les modifier ;
- lire en SQL un CSV, un JSON imbriqué et un fichier de logs, sans créer de table ;
- convertir des données brutes en Parquet, et mesurer le gain de place ;
- croiser plusieurs sources directement dans le lake ;
- démontrer les limites d'un data lake, qui justifient le lakehouse du TP3.

Prérequis. Avoir terminé le TP1 (environnement Python, `.venv`, DuckDB installés).

Le déroulé

Étape	Script	Ce qu'on fait	Zone du lake
Préparation	aucun	Créer le dossier et les fichiers sources	`sources/`
1. Ingestion	`lake1_ingestion.py`	Copier les fichiers tels quels, par source et par date	`lake/raw/`
2. Schema-on-read	`lake2_lecture.py`	Lire chaque format en SQL, sans table	lecture seule
3. Parquet	`lake3_parquet.py`	Convertir en format colonne, partitionner	`lake/processed/`

Étape	Script	Ce qu'on fait	Zone du lake
4. Analyse	`lake4_analyse.py`	Croiser commandes, avis et livraisons	lecture seule
5. Limites	`lake5_limites.py`	Démontrer ce que le lake ne sait pas faire	`lake/processed/`

Différence importante avec le TP1. Ici, il n'y a **pas de base de données** : pas de fichier `.duckdb`. Le lake n'est qu'un ensemble de dossiers et de fichiers. DuckDB ne sert que de **moteur de lecture** : il lit les fichiers, calcule, puis oublie tout. Les étapes restent à exécuter dans l'ordre, car chacune lit les fichiers produits par la précédente.

Règle de travail. À chaque étape : (1) lire l'objectif, (2) créer le script, (3) l'exécuter, (4) comparer au résultat attendu, (5) lire l'explication, (6) répondre aux questions.

Préparation

P.1 Ouvrir l'environnement du TP1

Ouvrir le projet `tp_lakehouse` dans PyCharm, puis le **Terminal** (`Alt + F12`). L'invite doit commencer par (`.venv`). Si ce n'est pas le cas, reprendre l'étape 0 du TP1 (configuration de l'interpréteur). Aucune nouvelle bibliothèque n'est nécessaire : DuckDB suffit.

P.2 Créer le dossier du TP2

Depuis le dossier `tp_lakehouse` :

```
mkdir exercice_lake
mkdir exercice_lake\sources
cd exercice_lake
```

Arborescence attendue à la fin du TP :

```
exercice_lake/
├── sources/                  <- ce que les systèmes du café nous envoient
│   ├── commandes.csv         (TP1)
│   ├── clients.csv           (TP1)
│   ├── produits.csv          (TP1)
│   ├── commandes_j2.csv      (nouveau : lot du lendemain)
│   ├── avis.json             (nouveau : avis de l'application)
│   └── livraisons.log        (nouveau : journal des coursiers)
├── lake1_ingestion.py … lake5_limites.py
└── lake/                     <- le data lake, créé par les scripts
    ├── raw/                  (données brutes, étape 1)
    └── processed/            (données en Parquet, étape 3)
```

Les commandes `python lakeX_....py` doivent toujours être lancées depuis le dossier `exercice_lake`. Avec « clic droit → Run » dans PyCharm, vérifier dans **Run → Edit Configurations** que le *Working directory* est `exercice_lake`.

P.3 Récupérer les données du TP1

Copier les trois fichiers du TP1 dans `sources` :

```
Copy-Item ..\exercice_etl\donnees\*.csv sources\
```
Vérifier avec `ls sources` : on doit voir `commandes.csv`, `clients.csv` et `produits.csv`.

P.4 Créer les trois nouvelles sources

Dans PyCharm, clic droit sur `sources` → **New → File**, puis coller le contenu exact.

`sources/commandes_j2.csv` : le lot de commandes reçu le lendemain. Attention, la commande 16 a déjà été envoyée dans le premier lot : la caisse l'a renvoyée par erreur.

```
commande_id,date_commande,client_id,produit_id,quantite,montant,canal
16,2025-03-08,1,3,1,12.500,sur place
17,2025-03-09,4,2,2,10.000,livraison
18,09/03/2025,6,5,1,9.000,Sur Place
19,2025-03-09,2,3,2,25.000,livraison
```
`sources/avis.json` : les avis clients de l'application. C'est une donnée **semi-structurée** : l'objet `client` est imbriqué, `tags` est une liste de longueur variable, et `commentaire` peut être vide (`null`).

```
[
  {"avis_id": 1, "commande_id": 2, "date": "2025-03-01", "note": 5,
   "commentaire": "Pizza encore chaude, parfait !",
   "client": {"id": 2, "appareil": "android"}, "tags": ["rapide", "chaud"]},
  {"avis_id": 2, "commande_id": 5, "date": "2025-03-03", "note": 3,
   "commentaire": "Bon mais livraison un peu longue",
   "client": {"id": 4, "appareil": "iphone"}, "tags": ["lent"]},
  {"avis_id": 3, "commande_id": 7, "date": "2025-03-04", "note": 2,
   "commentaire": "Arrivée froide",
   "client": {"id": 5, "appareil": "android"}, "tags": ["lent", "froid"]},
  {"avis_id": 4, "commande_id": 1, "date": "2025-03-01", "note": 5,
   "commentaire": null,
   "client": {"id": 1, "appareil": "web"}, "tags": []},
  {"avis_id": 5, "commande_id": 10, "date": "2025-03-05", "note": 4,
   "commentaire": "Salade très fraîche",
   "client": {"id": 1, "appareil": "iphone"}, "tags": ["frais"]},
  {"avis_id": 6, "commande_id": 13, "date": "2025-03-07", "note": 1,
   "commentaire": "Commande jamais arrivée...",
   "client": {"id": 6, "appareil": "android"}, "tags": ["annulé"]},
  {"avis_id": 7, "commande_id": 3, "date": "2025-03-02", "note": 4,
   "commentaire": "Croissants délicieux",
   "client": {"id": 3, "appareil": "web"}, "tags": ["bon goût"]}
]
```
`sources/livraisons.log` : le journal de l'application des coursiers. C'est une donnée **non structurée** : du texte, sans colonnes ni en-tête.

```
2025-03-01 12:41:05 | INFO | commande=2 | livreur=Sami | statut=LIVREE | duree=22min
2025-03-03 13:15:40 | WARN | commande=5 | livreur=Ines | statut=RETARD | duree=48min
2025-03-04 20:02:11 | WARN | commande=7 | livreur=Sami | statut=RETARD | duree=55min
2025-03-05 12:30:59 | INFO | commande=10 | livreur=Ines | statut=LIVREE | duree=19min
2025-03-07 19:47:23 | ERROR | commande=13 | livreur=Walid | statut=ANNULEE |
duree=0min
2025-03-09 12:10:02 | INFO | commande=17 | livreur=Walid | statut=LIVREE |
duree=25min
2025-03-09 13:05:47 | INFO | commande=19 | livreur=Sami | statut=LIVREE | duree=31min
```

P.5 Question d'observation

Pour chacun des six fichiers de `sources`, indiquer s'il est structuré, semi-structuré ou non structuré, et dire s'il pourrait être chargé tel quel dans le data warehouse du TP1.

Étape 1 : ingestion brute dans la zone raw

Objectif

Déposer tous les fichiers sources dans le data lake, **sans les ouvrir ni les modifier**, rangés par système source, par table et par date de réception.

Notion clé : la zone raw et l'organisation des dossiers

La zone `raw` (« brute ») est la mémoire du lake : elle garde une copie exacte de tout ce qui a été reçu, pour toujours. Dans un vrai projet, c'est un stockage objet (Amazon S3, Azure Data Lake, MinIO) ; ici, un simple dossier joue ce rôle.

Le rangement suit une convention :

```
lake/raw/<système source>/<table>/date_ingestion=<date du lot>/<fichier>
```

Le nom de dossier `date_ingestion=2025-03-08` respecte la **convention Hive** (`colonne=valeur`). Les outils comme DuckDB, Spark ou Trino savent transformer ce nom de dossier en colonne, comme on le verra à l'étape 2. Chaque nouveau lot crée un nouveau dossier : rien n'est jamais écrasé.

Démarche

1.1 Créer `lake1_ingestion.py` dans `exercice_lake` :

```
"""
ÉTAPE 1 : INGESTION BRUTE DANS LA ZONE RAW
On copie chaque fichier source TEL QUEL dans le data lake.
Rangement : lake/raw/<source>/<table>/date_ingestion=<date>/<fichier>
Aucune transformation, aucune base de données : seulement des dossiers et des
fichiers.
"""
import shutil
from pathlib import Path

RAW = Path("lake/raw")

# (fichier source, système source, table, date du lot d'ingestion)
LOTS = [
    # Lot 1 : reçu le 8 mars 2025
    ("sources/commandes.csv",    "caisse",    "commandes",  "2025-03-08"),
    ("sources/produits.csv",     "caisse",    "produits",   "2025-03-08"),
    ("sources/clients.csv",      "crm",       "clients",    "2025-03-08"),
    ("sources/avis.json",        "app",       "avis",       "2025-03-08"),
    ("sources/livraisons.log",   "coursiers", "livraisons", "2025-03-08"),
    # Lot 2 : reçu le 9 mars 2025 (nouvelles commandes seulement)
    ("sources/commandes_j2.csv", "caisse",    "commandes",  "2025-03-09"),
]
```

```
for fichier, source, table, date_lot in LOTS:
    dossier = RAW / source / table / f"date_ingestion={date_lot}"
    dossier.mkdir(parents=True, exist_ok=True)       # crée les dossiers s'ils
n'existent pas
    shutil.copy(fichier, dossier / Path(fichier).name)  # copie à l'identique, sans
rien modifier
    print(f"Ingéré : {fichier:28} -> {dossier}")

print("\n=== Contenu du data lake (zone raw) ===")
for f in sorted(RAW.rglob("*.*")):
    print(f"  {str(f):75} {f.stat().st_size:>6} octets")

nb = len(list(RAW.rglob("*.*")))
print(f"\n{nb} fichiers dans le lake. Formats présents :",
      sorted({f.suffix for f in RAW.rglob('*.*')}))
print("Étape 1 terminée. Les données brutes sont stockées, quel que soit leur
format.")
```

1.2 Exécuter :

```
python lake1_ingestion.py
```

1.3 Ouvrir le dossier `lake/raw` dans l'arborescence de PyCharm et parcourir les sous-dossiers.

Résultat attendu

```
=== Contenu du data lake (zone raw) ===
  lake/raw/app/avis/date_ingestion=2025-03-08/avis.json                    1258
octets
  lake/raw/caisse/commandes/date_ingestion=2025-03-08/commandes.csv          709
octets
  lake/raw/caisse/commandes/date_ingestion=2025-03-09/commandes_j2.csv       217
octets
  lake/raw/caisse/produits/date_ingestion=2025-03-08/produits.csv            199
octets
  lake/raw/coursiers/livraisons/date_ingestion=2025-03-08/livraisons.log     602
octets
  lake/raw/crm/clients/date_ingestion=2025-03-08/clients.csv                 208
octets

6 fichiers dans le lake. Formats présents : ['.csv', '.json', '.log']
```

Sous Windows, les chemins s'affichent avec des `\` au lieu des `/` : c'est normal. Les tailles peuvent varier de quelques octets selon les fins de ligne.

Explication du code

Instruction	Rôle
`LOTS = [...]`	Le « plan d'ingestion » : pour chaque fichier, sa source, sa table et la date du lot
`RAW / source / table / f"date_ingestion={date_lot}"`	Construit le chemin du dossier de destination
`mkdir(parents=True, exist_ok=True)`	Crée toute l'arborescence si besoin, sans erreur si

Instruction	Rôle
	elle existe déjà
`shutil.copy(...)`	Copie le fichier octet par octet : le contenu est strictement identique à la source
`RAW.rglob("*.*")`	Parcourt tous les fichiers du lake, dans tous les sous-dossiers
`f.suffix`	L'extension du fichier (`.csv`, `.json`, `.log`)

Différence avec le TP1. À l'étape 1 du TP1, on chargeait les fichiers dans des tables DuckDB. Ici, on se contente de copier des fichiers : il n'y a ni table, ni type, ni schéma. C'est ce qui permet d'accepter **n'importe quel format**, y compris les logs et le JSON imbriqué.

Questions

1. Pourquoi range-t-on les fichiers par date d'ingestion plutôt que d'écraser l'ancien fichier `commandes.csv` ?
2. Les doublons des commandes 6 et 12 et la date `09/03/2025` sont-ils corrigés à cette étape ? Pourquoi ?
3. Relancez le script. Combien de fichiers contient le lake ? Expliquez.
4. Le café reçoit demain des photos de ses plats (`.jpg`). Que faudrait-il ajouter au script pour les ingérer ?

Étape 2 : schema-on-read, lire chaque format en SQL

Objectif

Interroger les fichiers du lake directement en SQL, **sans créer de table** : un CSV réparti sur deux lots, un JSON imbriqué et un fichier de logs en texte libre.

Notion clé : schema-on-write contre schema-on-read

	Data warehouse (TP1)	Data lake (TP2)
Quand la structure est-elle définie ?	À l'écriture (*schema-on-write*)	À la lecture (*schema-on-read*)
Que se passe-t-il avec un fichier mal formé ?	Il est refusé au chargement	Il est accepté, le problème apparaît plus tard
Qui décide des colonnes et des types ?	Le concepteur du warehouse, une fois pour toutes	Chaque requête, au moment où elle lit

Démarche

2.1 Créer `lake2_lecture.py` :

```
"""
ÉTAPE 2 : SCHEMA-ON-READ
On lit les fichiers du lake directement en SQL, SANS créer de table.
La structure (colonnes, types) est déterminée au moment de la lecture.
"""
import duckdb

con = duckdb.connect()   # base en mémoire : rien n'est enregistré sur le disque

# ---------- 1. CSV : toutes les commandes, de tous les lots ----------
print("=== 1. Commandes : les deux lots lus d'un coup grâce au joker * ===")
con.sql("""
    SELECT date_ingestion, count(*) AS nb_lignes
    FROM read_csv('lake/raw/caisse/commandes/*/*.csv', hive_partitioning = true,
all_varchar = true)
    GROUP BY date_ingestion ORDER BY date_ingestion
""").show()
# hive_partitioning = true transforme le nom de dossier "date_ingestion=2025-03-08"
# en une vraie colonne date_ingestion.

print("=== Types devinés par DuckDB pour commandes.csv (sans all_varchar) ===")
con.sql("DESCRIBE SELECT * FROM
read_csv('lake/raw/caisse/commandes/*/*.csv')").show()

# ---------- 2. JSON imbriqué : les avis clients ----------
print("=== 2a. Avis : le JSON est lu tel quel, avec ses structures imbriquées ===")
con.sql("SELECT * FROM
read_json('lake/raw/app/avis/*/avis.json')").show(max_width=250)

print("=== 2b. On va chercher DANS l'objet client avec un point : client.appareil
===")
con.sql("""
    SELECT avis_id, commande_id, note, client.id AS client_id, client.appareil AS
appareil
    FROM read_json('lake/raw/app/avis/*/avis.json')
""").show()

print("=== 2c. On « déplie » la liste tags : une ligne par tag (unnest) ===")
con.sql("""
    SELECT tag, count(*) AS nb_avis
    FROM (SELECT unnest(tags) AS tag FROM read_json('lake/raw/app/avis/*/avis.json'))
    GROUP BY tag ORDER BY nb_avis DESC, tag
""").show()

# ---------- 3. Texte non structuré : les logs de livraison ----------
print("=== 3a. Logs lus comme du texte brut : une seule colonne ===")
con.sql("""
    SELECT line FROM read_csv('lake/raw/coursiers/livraisons/*/livraisons.log',
                              columns = {'line': 'VARCHAR'}, delim = '\\t', header =
false)
""").show(max_width=200)

print("=== 3b. Les mêmes logs, structurés à la lecture avec des expressions
régulières ===")
con.sql("""
```

```
    SELECT
        regexp_extract(line, '^(\\S+ \\S+)', 1)::TIMESTAMP      AS horodatage,
        regexp_extract(line, '\\| (\\w+) \\|', 1)               AS niveau,
        regexp_extract(line, 'commande=(\\d+)', 1)::INT         AS commande_id,
        regexp_extract(line, 'livreur=(\\w+)', 1)               AS livreur,
        regexp_extract(line, 'statut=(\\w+)', 1)                AS statut,
        regexp_extract(line, 'duree=(\\d+)min', 1)::INT         AS duree_min
    FROM read_csv('lake/raw/coursiers/livraisons/*/livraisons.log',
                  columns = {'line': 'VARCHAR'}, delim = '\\t', header = false)
""").show()

con.close()
print("Étape 2 terminée. Aucune table n'a été créée : la structure n'existe qu'au
moment de la lecture.")
```

2.2 Exécuter :

```
python lake2_lecture.py
```

Résultat attendu

- **Partie 1** : deux lignes, `2025-03-08 → 18` et `2025-03-09 → 4`. Puis la liste des colonnes devinées : `commande_id`, `quantite` en `BIGINT`, `montant` en `DOUBLE`, mais `date_commande` en `VARCHAR` (à cause des deux formats), et une colonne `date_ingestion` de type `DATE`, tirée du nom de dossier.
- **Partie 2a** : les 7 avis, avec une colonne `client` de type `struct(id bigint, appareil varchar)` et une colonne `tags` de type `varchar[]` (une liste).
- **Partie 2b** : les mêmes avis, avec `client_id` et `appareil` sortis de l'objet imbriqué.
- **Partie 2c** : le tag `lent` arrive en tête (2 avis).
- **Partie 3a** : les 7 lignes du log, dans une seule colonne `line`.
- **Partie 3b** : les mêmes lignes découpées en 6 colonnes typées (`horodatage` en `timestamp`, `commande_id` et `duree_min` en `int32`).

Explication du code

Le joker et les partitions Hive. `'lake/raw/caisse/commandes/*/*.csv'` signifie « tous les fichiers `.csv` de tous les sous-dossiers de `commandes` ». Les deux lots sont lus comme un seul tableau. Avec `hive_partitioning = true`, DuckDB lit le nom de dossier `date_ingestion=2025-03-08` et en fait une colonne : on sait ainsi de quel lot vient chaque ligne.

L'inférence de types. Sans `all_varchar`, DuckDB lit un échantillon du fichier et **devine** le type de chaque colonne. C'est le principe du schema-on-read : personne n'a déclaré ces types, ils sont déduits à chaque lecture. `date_commande` reste du texte car les valeurs `02/03/2025` ne ressemblent pas à une date ISO.

Le JSON imbriqué.

- `client` devient un `struct` : un objet avec des champs. On accède à un champ avec un point, `client.appareil`, comme en Python.
- `tags` devient une liste. `unnest(tags)` la « déplie » : un avis avec 2 tags produit 2 lignes. L'avis 4, dont la liste est vide, ne produit aucune ligne.

Les logs. Le fichier n'a pas de colonnes. On le lit comme un CSV d'une seule colonne `line`, avec un séparateur (`delim = '\t'`, la tabulation) qui n'apparaît jamais dans le texte : chaque ligne reste entière. Ensuite, `regexp_extract(texte, motif, 1)` extrait la partie du texte qui correspond au motif entre parenthèses :

Motif	Signification	Exemple extrait
`^(\S+ \S+)`	Au début de la ligne, deux mots séparés par un espace	`2025-03-01 12:41:05`
`commande=(\d+)`	Les chiffres qui suivent `commande=`	`2`
`livreur=(\w+)`	Le mot qui suit `livreur=`	`Sami`
`duree=(\d+)min`	Les chiffres entre `duree=` et `min`	`22`

Dans le code Python, les antislashs sont doublés (`\\d`) : en Python, `\\` produit un seul `\` dans la chaîne envoyée à DuckDB.

Questions

1. Rien n'a été enregistré sur le disque par ce script. Où se trouvent alors les « tables » que l'on vient d'interroger ?
2. Pourquoi `date_commande` est-elle devinée comme du texte (`VARCHAR`) et non comme une date ?
3. Que se passerait-il à la partie 3b si un coursier écrivait `duree=environ 20min` ?
4. Écrivez une requête qui compte les avis par `appareil` (android, iphone, web).

Étape 3 : zone processed en Parquet

Objectif

Créer une deuxième zone dans le lake, `lake/processed`, où les données brutes sont converties au format **Parquet** : structurées, typées et compressées. On mesure ensuite le gain de place sur un petit puis sur un gros volume.

Notion clé : le format Parquet

Un fichier CSV stocke les données **ligne par ligne**, sous forme de texte. Un fichier Parquet les stocke **colonne par colonne**, en binaire, avec trois avantages :

- **compression** : les valeurs d'une même colonne se ressemblent et se compressent très bien ;
- **types enregistrés** : le fichier sait qu'une colonne est une date ou un entier, plus besoin de deviner ;
- **lecture sélective** : pour calculer `sum(montant)`, un moteur ne lit que la colonne `montant`.

Attention : la zone `processed` n'est **pas nettoyée** comme le schema `propre` du TP1. On change seulement le format, pas le contenu : les doublons et les erreurs restent.

Démarche

3.1 Créer `lake3_parquet.py` :

```
"""
ÉTAPE 3 : ZONE PROCESSED EN PARQUET
On convertit les données brutes en Parquet (format en colonnes, compressé et typé),
dans une deuxième zone du lake : lake/processed/.
La zone raw n'est jamais modifiée.
"""
from pathlib import Path

import duckdb

con = duckdb.connect()
PROC = Path("lake/processed")
PROC.mkdir(parents=True, exist_ok=True)

LOGS = """read_csv('lake/raw/coursiers/livraisons/*/livraisons.log',
                   columns = {'line': 'VARCHAR'}, delim = '\\t', header = false)"""

# ---------- 1. Commandes : tous les lots, partitionnés par date d'ingestion --------
--
con.sql("""
    COPY (SELECT * FROM read_csv('lake/raw/caisse/commandes/*/*.csv',
                                 hive_partitioning = true, all_varchar = true))
    TO 'lake/processed/commandes' (FORMAT parquet, PARTITION_BY (date_ingestion),
OVERWRITE_OR_IGNORE)
""")

# ---------- 2. Avis : le JSON imbriqué devient un tableau à plat ----------
con.sql("""
    COPY (SELECT avis_id, commande_id, date::DATE AS date_avis, note, commentaire,
                 client.id AS client_id, client.appareil AS appareil, tags
          FROM read_json('lake/raw/app/avis/*/avis.json'))
    TO 'lake/processed/avis.parquet' (FORMAT parquet)
""")

# ---------- 3. Livraisons : le texte devient un tableau typé ----------
con.sql(f"""
    COPY (SELECT regexp_extract(line, '^(\\S+ \\S+)', 1)::TIMESTAMP AS horodatage,
                 regexp_extract(line, 'commande=(\\d+)', 1)::INT    AS commande_id,
                 regexp_extract(line, 'livreur=(\\w+)', 1)          AS livreur,
                 regexp_extract(line, 'statut=(\\w+)', 1)           AS statut,
                 regexp_extract(line, 'duree=(\\d+)min', 1)::INT    AS duree_min
          FROM {LOGS})
    TO 'lake/processed/livraisons.parquet' (FORMAT parquet)
""")

# ---------- 4. Clients et produits ----------
for source, table in [("crm", "clients"), ("caisse", "produits")]:
    con.sql(f"""COPY (SELECT * FROM read_csv('lake/raw/{source}/{table}/*/*.csv',
all_varchar = true))
                TO 'lake/processed/{table}.parquet' (FORMAT parquet)""")

print("=== Contenu de la zone processed ===")
for f in sorted(PROC.rglob("*.parquet")):
    print(f"  {str(f):70} {f.stat().st_size:>6} octets")
```

```
print("\n=== Lecture d'un Parquet : les types sont enregistrés DANS le fichier ===")
con.sql("DESCRIBE SELECT * FROM 'lake/processed/livraisons.parquet'").show()

# ---------- 5. Comparaison de taille : petit fichier puis gros volume ----------
csv_small = sum(f.stat().st_size for f in
Path("lake/raw/caisse/commandes").rglob("*.csv"))
pq_small = sum(f.stat().st_size for f in
Path("lake/processed/commandes").rglob("*.parquet"))
print(f"Commandes (18 + 4 lignes) : CSV = {csv_small} octets | Parquet = {pq_small}
octets")

# On fabrique 200 000 commandes pour voir l'effet sur un vrai volume
Path("lake/test_volume").mkdir(parents=True, exist_ok=True)
con.sql("""
    CREATE TABLE gros AS
    SELECT i AS commande_id, DATE '2025-01-01' + (i % 365)::INT AS date_commande,
           (i % 6) + 1 AS client_id, (i % 5) + 1 AS produit_id, (i % 4) + 1 AS
quantite,
           round(((i % 5) + 1) * 3.5, 3) AS montant,
           CASE WHEN i % 3 = 0 THEN 'livraison' ELSE 'sur place' END AS canal
    FROM range(200000) t(i)
""")
con.sql("COPY gros TO 'lake/test_volume/commandes.csv' (HEADER)")
con.sql("COPY gros TO 'lake/test_volume/commandes.parquet' (FORMAT parquet)")
csv_big = Path("lake/test_volume/commandes.csv").stat().st_size
pq_big = Path("lake/test_volume/commandes.parquet").stat().st_size
print(f"Commandes (200 000 lignes) : CSV = {csv_big:,} octets | Parquet = {pq_big:,}
octets "
      f"-> Parquet {csv_big / pq_big:.1f} fois plus petit")

con.close()
print("\nÉtape 3 terminée. La zone processed contient des fichiers Parquet typés et
compressés.")
```

3.2 Exécuter :

```
python lake3_parquet.py
```

Résultat attendu

```
=== Contenu de la zone processed ===
  lake/processed/avis.parquet                                          1710 octets
  lake/processed/clients.parquet                                       1071 octets
  lake/processed/commandes/date_ingestion=2025-03-08/data_0.parquet    1466 octets
  lake/processed/commandes/date_ingestion=2025-03-09/data_0.parquet    1130 octets
  lake/processed/livraisons.parquet                                     927 octets
  lake/processed/produits.parquet                                      1061 octets
```

Puis la description de `livraisons.parquet` (`horodatage` en `TIMESTAMP`, `commande_id` et `duree_min` en `INTEGER`), et enfin :

```
Commandes (18 + 4 lignes) : CSV = 926 octets | Parquet = 2596 octets
Commandes (200 000 lignes) : CSV = 7,608,960 octets | Parquet = 1,016,583 octets ->
Parquet 7.5 fois plus petit
```

Les tailles exactes peuvent varier légèrement selon la version de DuckDB.

Explication du code

`COPY (requête) TO 'fichier' (FORMAT parquet)` exécute la requête et écrit le résultat dans un fichier Parquet. C'est l'instruction centrale de cette étape.

`PARTITION_BY (date_ingestion)` écrit un sous-dossier par valeur de la colonne : `date_ingestion=2025-03-08/` et `date_ingestion=2025-03-09/`. Une requête qui filtre sur une date ne lira que le dossier concerné. `OVERWRITE_OR_IGNORE` permet de relancer le script sans erreur.

Avis et livraisons « mis à plat ». On reprend les requêtes de l'étape 2 (`client.appareil`, `regexp_extract`), mais leur résultat est maintenant **enregistré**. Les analystes n'auront plus besoin de redécouper les logs à chaque lecture.

La comparaison de taille. Sur 22 lignes, le Parquet est **plus gros** que le CSV : chaque fichier Parquet contient un en-tête de métadonnées (schéma, statistiques), qui pèse plus lourd que les données elles-mêmes. Sur 200 000 lignes, la compression l'emporte nettement : le Parquet est environ **7,5 fois plus petit**. Parquet est fait pour les gros volumes.

Questions

1. Ouvrez `lake/processed/avis.parquet` dans PyCharm. Pourquoi ne peut-on pas le lire comme un fichier texte ?
2. Les doublons des commandes 6, 12 et 16 sont-ils encore présents dans `lake/processed/commandes` ? Comment le vérifier en SQL ?
3. Pourquoi le Parquet est-il plus gros que le CSV sur les petites données ?
4. Quel est l'intérêt d'avoir gardé les fichiers d'origine dans `raw` alors qu'ils existent maintenant en Parquet ?

Étape 4 : analyses croisées directement sur le lake

Objectif

Répondre à des questions que le data warehouse du TP1 ne pouvait pas traiter, en croisant trois sources de formats différents : les commandes (CSV), les avis (JSON) et les livraisons (logs).

Notion clé : le nettoyage à chaque lecture

Le lake ne nettoie jamais les données. Chaque analyste doit donc **refaire lui-même** le nettoyage au moment de sa requête. Pour éviter de réécrire ce code partout, on utilise une **vue** : une requête à laquelle on donne un nom. Une vue ne stocke aucune donnée ; elle est réexécutée à chaque fois qu'on l'utilise.

Démarche

4.1 Créer `lake4_analyse.py` :

```
"""
ÉTAPE 4 : ANALYSES CROISÉES DIRECTEMENT SUR LE LAKE
On répond aux questions du gérant en croisant commandes, avis et livraisons.
Attention : les commandes de la zone processed ne sont PAS nettoyées.
Chaque requête doit donc refaire elle-même le nettoyage nécessaire.
```

```
"""
import duckdb

con = duckdb.connect()

# Une "vue" n'enregistre aucune donnée : c'est seulement une requête à laquelle on
donne un nom.
# Le nettoyage du TP1 est réécrit ici, car le lake ne l'a jamais fait.
con.sql("""
    CREATE VIEW commandes AS
    SELECT DISTINCT
        commande_id::INT AS commande_id,
        coalesce(try_strptime(date_commande, '%Y-%m-%d'),
                 try_strptime(date_commande, '%d/%m/%Y'))::DATE AS date_commande,
        client_id::INT AS client_id, produit_id::INT AS produit_id,
        quantite::INT AS quantite, montant::DOUBLE AS montant,
        lower(trim(canal)) AS canal
    FROM 'lake/processed/commandes/*/*.parquet'
    WHERE montant IS NOT NULL AND quantite::INT > 0
""")
con.sql("CREATE VIEW avis       AS SELECT * FROM 'lake/processed/avis.parquet'")
con.sql("CREATE VIEW livraisons AS SELECT * FROM
'lake/processed/livraisons.parquet'")
con.sql("CREATE VIEW produits   AS SELECT produit_id::INT AS produit_id, nom_produit,
categorie "
        "FROM 'lake/processed/produits.parquet'")

print("=== Question 1 : note moyenne des clients par produit (commandes + avis +
produits) ===")
con.sql("""
    SELECT p.nom_produit, count(a.avis_id) AS nb_avis, round(avg(a.note), 2) AS
note_moyenne
    FROM avis a
    JOIN commandes c USING (commande_id)
    JOIN produits p  USING (produit_id)
    GROUP BY p.nom_produit ORDER BY note_moyenne DESC
""").show()

print("=== Question 2 : la durée de livraison influence-t-elle la note ? (livraisons
+ avis) ===")
con.sql("""
    SELECT l.commande_id, l.livreur, l.statut, l.duree_min, a.note, a.commentaire
    FROM livraisons l
    LEFT JOIN avis a USING (commande_id)
    ORDER BY l.duree_min DESC
""").show(max_width=200)

print("=== Question 3 : performance des livreurs ===")
con.sql("""
    SELECT livreur,
           count(*)                                          AS nb_courses,
           count(*) FILTER (WHERE statut = 'RETARD')     AS nb_retards,
           count(*) FILTER (WHERE statut = 'ANNULEE')    AS nb_annulees,
           round(avg(duree_min) FILTER (WHERE statut <> 'ANNULEE'), 1) AS
duree_moyenne_min
```

```
    FROM livraisons GROUP BY livreur ORDER BY nb_courses DESC
""").show()

print("=== Question 4 : chiffre d'affaires par jour, lots 1 et 2 réunis ===")
con.sql("""
    SELECT date_commande, count(*) AS nb_commandes, round(sum(montant), 3) AS ca
    FROM commandes GROUP BY date_commande ORDER BY date_commande
""").show()

print("=== Question 5 : que disent les clients mécontents ? (texte libre) ===")
con.sql("""
    SELECT note, commentaire, tags FROM avis WHERE note <= 3 ORDER BY note
""").show(max_width=200)

con.close()
print("Étape 4 terminée. On a croisé trois sources de formats différents, sans base de données.")
```

4.2 Exécuter :

```
python lake4_analyse.py
```

Résultat attendu

Question	Résultat
1. Note par produit	Café express 5,0 · Croissant 4,0 · Salade tunisienne 3,5 · Pizza margherita 2,67 (3 avis)
2. Durée et note	Les deux livraisons en retard (55 et 48 min) ont les notes 2 et 3 ; les livraisons de 19 et 22 min ont 4 et 5 ; la commande annulée a la note 1
3. Livreurs	Sami : 3 courses, 1 retard, 36,0 min en moyenne · Ines : 2 courses, 1 retard, 33,5 min · Walid : 2 courses, 1 annulation, 25,0 min
4. CA par jour	9 jours, du 1er au 9 mars ; le 9 mars : 3 commandes pour 44,0 DT ; la commande 16 n'est comptée qu'une fois
5. Clients mécontents	3 avis (notes 1, 2 et 3) : « jamais arrivée », « arrivée froide », « livraison un peu longue »

Explication du code

Les vues. `CREATE VIEW commandes AS SELECT ...` donne un nom à une requête. Ensuite, on peut écrire `FROM commandes` comme s'il s'agissait d'une table. La vue `commandes` reprend les règles de nettoyage du TP1 : `DISTINCT` (doublons), `try_strptime` (dates), `lower(trim(...))` (canal), et les filtres sur le montant et la quantité.

Pourquoi la commande 16 n'est comptée qu'une fois. Elle existe dans les deux lots, mais la vue ne sélectionne pas la colonne `date_ingestion`. Les deux copies deviennent donc identiques, et `DISTINCT` en supprime une. Si l'une des deux copies avait un montant différent, `DISTINCT` garderait les deux : le lake ne sait pas quelle version est la bonne.

Les jointures entre formats. La question 1 joint un fichier issu d'un JSON (avis), un fichier issu d'un CSV (commandes) et un troisième (produits), grâce à la clé commune `commande_id` puis `produit_id`. La question 2 joint les logs aux avis avec un `LEFT JOIN` : les livraisons sans avis (commandes 17 et 19) restent affichées, avec une note `NULL`.

Le texte libre. La question 5 montre l'intérêt du lake : on garde les commentaires et les tags, qui expliquent **pourquoi** une note est basse. Un schéma en étoile classique ne les aurait pas conservés.

Questions

1. Quelle conclusion le gérant peut-il tirer de la question 2 ?
2. Supprimez le mot `DISTINCT` dans la vue `commandes`, relancez le script : que devient le CA du 3, du 6 et du 8 mars ? Expliquez, puis remettez `DISTINCT`.
3. Un deuxième analyste veut le CA par canal. Que doit-il recopier dans son propre script ? Quel est le risque ?
4. Écrivez une requête qui donne, pour chaque livreur, la note moyenne des clients qu'il a livrés.

Étape 5 : les limites du data lake

Objectif

Démontrer, avec du code, cinq choses qu'un data lake « simple » (des fichiers dans des dossiers) **ne sait pas faire**. Ces limites expliquent pourquoi on a inventé le lakehouse, objet du TP3.

Toutes les démonstrations travaillent dans un dossier de test, `lake/test_limites`, pour ne pas abîmer le lake construit aux étapes précédentes. On peut donc relancer le script autant de fois qu'on veut.

Démarche

5.1 Créer `lake5_limites.py` :

```
"""
ÉTAPE 5 : LES LIMITES DU DATA LAKE
On démontre, une par une, cinq choses qu'un simple data lake ne sait pas faire.
Chaque démonstration travaille sur une COPIE, pour ne pas abîmer le lake des étapes
précédentes.
"""
import shutil
from pathlib import Path

import duckdb

con = duckdb.connect()
TEST = Path("lake/test_limites")
shutil.rmtree(TEST, ignore_errors=True)       # on repart d'un dossier de test vide
TEST.mkdir(parents=True)
shutil.copy("lake/processed/avis.parquet", TEST / "avis.parquet")
T = TEST.as_posix()                            # chemin avec des "/" : fonctionne
aussi sous Windows

# ---------- Limite 1 : les erreurs ne sont jamais corrigées ----------
print("=== Limite 1 : doublons et erreurs restent dans le lake ===")
```

```
con.sql("""
    SELECT commande_id, count(*) AS nb_fois, list(date_ingestion) AS lots
    FROM 'lake/processed/commandes/*/*.parquet'
    GROUP BY commande_id HAVING count(*) > 1 ORDER BY commande_id::INT
""").show()
print("  La commande 16 a été envoyée dans les deux lots : le lake garde les deux
copies.\n")

# ---------- Limite 2 : impossible de modifier une ligne ----------
print("=== Limite 2 : pas de UPDATE ni de DELETE sur un fichier Parquet ===")
try:
    con.sql(f"UPDATE '{T}/avis.parquet' SET note = 4 WHERE avis_id = 6")
except Exception as e:
    print("  Erreur :", str(e).splitlines()[0][:90])
print("  Pour corriger UNE note, il faut relire tout le fichier et le réécrire en
entier :")
con.sql(f"""COPY (SELECT * REPLACE (CASE WHEN avis_id = 6 THEN 4 ELSE note END AS
note)
                FROM '{T}/avis.parquet')
           TO '{T}/avis_corrige.parquet' (FORMAT parquet)""")
print("  -> nouveau fichier avis_corrige.parquet écrit (l'ancien existe toujours à
côté).\n")

# ---------- Limite 3 : pas d'historique, un écrasement est définitif ----------
print("=== Limite 3 : pas d'historique ===")
avant = con.sql(f"SELECT count(*) FROM '{T}/avis.parquet'").fetchall()[0][0]
con.sql(f"""COPY (SELECT * FROM '{T}/avis.parquet' WHERE note >= 4)
           TO '{T}/avis.parquet' (FORMAT parquet)""")      # erreur de manipulation :
on écrase
apres = con.sql(f"SELECT count(*) FROM '{T}/avis.parquet'").fetchall()[0][0]
print(f"  Avis avant l'écrasement : {avant} | après : {apres}")
print("  Les avis négatifs ont disparu. Aucune commande ne permet de revenir en
arrière.\n")

# ---------- Limite 4 : aucun contrôle du schéma à l'écriture ----------
print("=== Limite 4 : le lake accepte n'importe quel fichier, même incohérent ===")
mauvais = TEST / "commandes_j3.csv"
mauvais.write_text("commande_id,date_commande,client_id,produit_id,quantite,montant,c
anal\n"
                   "20,2025-03-10,3,1,deux,douze dinars,sur place\n", encoding="utf-
8")
shutil.copy("lake/raw/caisse/commandes/date_ingestion=2025-03-09/commandes_j2.csv",
TEST / "commandes_j2.csv")
print("  Le fichier commandes_j3.csv (quantité 'deux', montant 'douze dinars') a été
déposé sans erreur.")
try:
    con.sql(f"SELECT sum(montant) FROM read_csv('{T}/commandes_j*.csv')").show()
except Exception as e:
    print("  Mais la lecture échoue plus tard, chez l'analyste :")
    print("  Erreur :", str(e).splitlines()[0][:95])
    print("  -> la colonne montant contient du texte : DuckDB ne peut plus en faire
la somme.")
print()

# ---------- Limite 5 : pas de transaction, lectures incohérentes ----------
```

```
print("=== Limite 5 : pas de transaction ===")
print("  Une écriture Parquet n'est pas atomique : si le programme plante au
milieu,")
print("  ou si un analyste lit le dossier pendant qu'un autre programme y écrit,")
print("  il peut voir un fichier à moitié écrit, ou un mélange de l'ancien et du
nouveau.\n")

# ---------- Bilan : risque de data swamp ----------
print("=== Bilan : à quoi ressemble le lake maintenant ? ===")
for f in sorted(Path("lake").rglob("*.*")):
    print("  ", f)
print("\n  Quels fichiers sont fiables ? Lequel est la bonne version des avis ?")
print("  Sans catalogue ni règles, le lake devient un « data swamp » (marécage de
données).")

con.close()
print("\nÉtape 5 terminée. Ces limites sont exactement ce que le lakehouse (TP3) va
résoudre.")
```

5.2 Exécuter :

```
python lake5_limites.py
```

Résultat attendu

```
=== Limite 1 : doublons et erreurs restent dans le lake ===
│ 6           │       2 │ [2025-03-08, 2025-03-08] │
│ 12          │       2 │ [2025-03-08, 2025-03-08] │
│ 16          │       2 │ [2025-03-08, 2025-03-09] │

=== Limite 2 : pas de UPDATE ni de DELETE sur un fichier Parquet ===
  Erreur : Binder Error: Can only update base table

=== Limite 3 : pas d'historique ===
  Avis avant l'écrasement : 7 | après : 4

=== Limite 4 : le lake accepte n'importe quel fichier, même incohérent ===
  Le fichier commandes_j3.csv (...) a été déposé sans erreur.
  Mais la lecture échoue plus tard, chez l'analyste :
  Erreur : Binder Error: No function matches the given name and argument types
'sum(VARCHAR)'...
```

Puis le texte de la limite 5 et la liste de tous les fichiers du lake (18 fichiers répartis dans `raw`, `processed`, `test_volume` et `test_limites`).

Explication des cinq limites

Limite	Ce que montre le script	Pourquoi c'est un problème
1. Pas de qualité garantie	Les doublons 6 et 12 du fichier, et la commande 16 reçue dans deux lots, sont toujours là	Chaque analyste doit penser à dédoublonner ; un oubli fausse le CA
2. Pas de modification	`UPDATE` est refusé sur un fichier Parquet ; il faut réécrire tout le fichier	Corriger une seule erreur coûte très cher sur de gros fichiers, et crée des

Limite	Ce que montre le script	Pourquoi c'est un problème
de ligne		copies
3. Pas d'historique	Un `COPY` qui écrase le fichier fait passer les avis de 7 à 4, sans retour possible	Une erreur de manipulation détruit des données définitivement
4. Pas de contrôle du schéma	Un fichier avec `deux` et `douze dinars` est accepté sans erreur ; c'est la lecture qui échoue	L'erreur est découverte tard, par quelqu'un qui ne l'a pas causée
5. Pas de transaction	Une écriture n'est pas « tout ou rien »	Un lecteur peut voir des données à moitié écrites

Le « data swamp ». Le bilan affiche maintenant 18 fichiers, dont `avis.parquet` en trois versions (dans `processed`, dans `test_limites`, et `avis_corrige.parquet`). Sans catalogue qui documente chaque fichier, son propriétaire et sa fiabilité, plus personne ne sait quelle version utiliser. Un lake qui grossit ainsi sans règles devient un **marécage de données** : tout est stocké, mais rien n'est exploitable en confiance.

Explication de quelques instructions.

- `list(date_ingestion)` regroupe dans une liste toutes les valeurs d'un groupe : on voit de quels lots vient chaque doublon.
- `SELECT * REPLACE (... AS note)` reprend toutes les colonnes, sauf `note` qui est remplacée par une nouvelle expression.
- `try ... except` intercepte l'erreur pour l'afficher proprement au lieu d'arrêter le script.
- `TEST.as_posix()` écrit le chemin avec des `/`, ce qui évite les problèmes d'antislash sous Windows.

Questions

1. Pour chacune des cinq limites, donnez un exemple concret de conséquence pour le gérant du Café Jasmin.
2. Dans le data warehouse du TP1, le fichier `commandes_j3.csv` aurait-il pu être chargé ? À quelle étape aurait-il été rejeté ?
3. La zone `raw` est-elle concernée par la limite 3 ? Que devrait-on interdire pour la protéger ?
4. Proposez deux règles d'organisation simples pour éviter que ce lake devienne un data swamp.

Synthèse, exercices et corrigé

Bilan : data warehouse (TP1) ou data lake (TP2) ?

Complétez ce tableau à partir de ce que vous avez **observé** dans les deux TP.

Critère	Data warehouse (TP1)	Data lake (TP2)
Peut stocker des logs et du JSON imbriqué tels quels		

Critère	Data warehouse (TP1)	Data lake (TP2)
Les données sont nettoyées une fois pour toutes		
La structure est définie à l'écriture ou à la lecture		
Un fichier mal formé est refusé dès son arrivée		
On peut modifier une seule ligne		
On peut revenir à une version passée		
Coût de stockage		
Utilisateurs les mieux servis		

Exercices de manipulation

Pour chaque exercice, modifiez ce qui est indiqué, relancez les scripts nécessaires et notez ce que vous observez.

1. **Un troisième lot.** Créez `sources/commandes_j3.csv` avec deux nouvelles commandes du 10 mars (numéros 20 et 21). Ajoutez une ligne dans `LOTS` avec la date `2025-03-10`, puis relancez les étapes 1, 3 et 4. Combien de dossiers `date_ingestion=` contient maintenant `lake/raw/caisse/commandes` ? Le 10 mars apparaît-il dans la question 4 ?
2. **Un nouvel avis.** Ajoutez dans `avis.json` un avis 8 sur la commande 17 (note 5, appareil `iphone`, tags `["rapide"]`). Relancez les étapes 1, 3 et 4. Quelle note moyenne obtient maintenant Walid à la question 4 de l'étape 4 ?
3. **Une nouvelle colonne.** Ajoutez une colonne `pourboire` à la fin de `commandes_j3.csv` (exercice 1). Relancez l'étape 2 : comment DuckDB réagit-il à des fichiers qui n'ont pas les mêmes colonnes ? (Indice : ajoutez `union_by_name = true` dans `read_csv`.)
4. **Un format inattendu.** Placez une image quelconque (`.jpg`) dans `lake/raw/app/photos/date_ingestion=2025-03-08/`. Relancez l'étape 1. Le lake l'accepte-t-il ? Peut-on l'interroger en SQL ?

Compte rendu à rendre

- Les réponses aux questions de la préparation (P.5) et des étapes 1 à 5.
- Une capture d'écran du résultat de chaque script.
- Une capture de l'arborescence `lake/` dans PyCharm à la fin du TP.
- Le tableau de bilan complété.
- Les observations des exercices de manipulation.